THE SYNTAX SURGEON- AI: INTELLIGENT PYTHON CODE GENERATION ASSISTANT.

Project Report

Submitted to **Manonmaniam Sundaranar** University in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

NAAN MUDHALVAN COURSE

Submitted By

ANTO SUBIKSHA A	**23081021802112006**
BENEZIA JUDITH J	**23081021802112009**
FAJULA DAWOOD P	**23081021802112011**
MOHAMED RASIHA S A	**23081021802112019**

Under the guidance of

Mrs. Dr. A. Aaisha Nazleem

MCA., M.Phil.,Ph.D mam
Assistant Professor,

DEPARTMENT OF COMPUTER SCIENCE

ANNAI HAJIRA WOMEN'S COLLEGE (AFFILIATED TO MANONMANIAM SUNDARANAR UNIVERSITY)

TIRUNELVELI – 627005

MARCH– 2026

CERTIFICATE

This is to certify that the project entitled as **"DOC GENIE – PYTHON FUNCTION DOCSTRING GENERATOR"** is a bonafide work done by _____, _____, _____, _____

for the **Naan Mudhalvan Course (Agentic AI Application Development)** in partial fulfillment of the requirements for the award of the degree in Bachelor of Computer Science during the academic year (VI Semester) 2025-2026.

Place:

Date:

Signature of the H.O.D **SPOC**

Submitted for the Naan Mudhalvan university Examination held on _____ at the Department of Computer Science, Annai Hajira Women's College, Tirunelveli.

Principal **External Examiner**

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
1.	**ABSTRACT**	1
2.	**INTRODUCTION**	2
3.	**SYSTEM REQUIREMENTS** 3.1 HARDWARE REQUIREMENTS 3.2 SOFTWARE REQUIREMENTS 3.3 CORE TECHNOLOGIES	4
4.	**SYSTEM DESIGN** 4.1 TECHNICAL ARCHITECTURE 4.2 PROJECT STRUCTURE	5
5.	**PROJECT WORKFLOW**	12
6.	**ALGORITHM**	14
7.	**IMPLEMENTATION** 7.1 CODING 7.2 OUTPUT	15
8.	**CONCLUSION**	32

1. ABSTRACT

The Syntax Surgeon project focuses on developing a supportive system that helps users identify and correct mistakes in programming code while also providing answers to their questions through a conversational interface. Writing programs can be challenging, especially for students and beginners who are still learning programming concepts. Even a small mistake in the code can cause errors that stop the program from running properly. Finding these mistakes and correcting them can take a lot of time and may create frustration for learners. This project aims to simplify that process by providing a system that can assist users in detecting errors and suggesting corrected versions of their code in an easy and understandable way. The system allows users to enter or paste their programming code and receive immediate suggestions for correcting common mistakes. In addition to fixing code, the system also supports a question-and-answer style interaction where users can ask questions and receive explanations. This makes the platform not only a problem-solving tool but also a learning support system. By receiving explanations along with corrected code, users can better understand what went wrong and how to avoid similar mistakes in the future. This helps improve programming skills over time and encourages users to learn through practice and interaction. Another useful aspect of the project is its ability to maintain a record of conversations between the user and the system. This allows users to go back and review earlier questions, explanations, and corrections whenever they want. Keeping track of these interactions can be helpful for learning, revision, and documentation purposes. The stored conversation can also be shared or saved for future reference, which can support collaborative learning or team discussions when working on programming tasks. The design of the system focuses on simplicity and ease of use so that users with basic programming knowledge can use it comfortably. The interface is designed to allow smooth communication between the user and the system, making the interaction feel natural and efficient. This makes the system suitable for students, beginners, programmers, and even teams working together on coding tasks. By reducing the effort required to locate and fix programming mistakes, the system helps users complete their work more quickly and effectively. Overall, the Syntax Surgeon project demonstrates how an intelligent assistant can make programming support more accessible and convenient. By combining error correction, interactive communication, and learning support in one platform, the system provides a helpful environment for users to improve their coding abilities and solve problems more efficiently. education and simplifying the development process.

2.INTRODUCTION

Programming has become one of the most important skills in the modern technological world. Many applications, websites, and digital services are developed using programming languages. However, writing programs is not always easy, especially for beginners who are still learning the rules and structure of coding. Even a small mistake in the program can cause errors that prevent the program from running correctly. These mistakes may occur due to missing symbols, incorrect formatting, typing errors, or misunderstanding of programming concepts. Because of this, programmers often spend a large amount of time identifying and correcting errors in their code.

In the learning stage, students and beginners frequently face difficulties while understanding why their code is not working properly. Sometimes the error messages shown by programming tools can be confusing or difficult to understand. As a result, learners may struggle to find the exact problem in their program. This can slow down the learning process and reduce confidence among new programmers. Therefore, there is a strong need for supportive systems that can guide users in identifying mistakes, correcting them, and understanding the correct way to write programs.

The Syntax Surgeon project is developed to address these challenges by providing a system that can assist users in detecting and correcting errors in programming code. The system acts as an intelligent assistant that interacts with users through a simple chat-based interface. Users can enter their programming code or ask questions, and the system responds with helpful suggestions and explanations. By providing corrected versions of the code along with explanations, the system helps users understand what went wrong and how the issue can be fixed. This not only solves the immediate problem but also helps users improve their coding knowledge over time.

Another important feature of this system is its ability to automatically recognize when a user enters programming code. Once the code is detected, the system analyses it to find possible mistakes and formatting problems. After identifying the issues, it provides a corrected version of the code to the user. This process reduces the time spent on debugging and helps programmers quickly fix errors in their programs. Such assistance is especially useful for beginners who may not yet be familiar with common programming mistakes.

The project also supports interactive communication between the user and the system. Instead of simply correcting code, the system allows users to ask questions and receive explanations about programming concepts. This conversational approach makes the learning experience

more engaging and easier to understand. Users can continue asking questions until they clearly understand the concept or the solution to their problem. This feature makes the system useful not only as a problem-solving tool but also as a learning assistant.

Another advantage of the Syntax Surgeon system is that it keeps track of the conversation between the user and the assistant. This conversation history allows users to review their previous questions, solutions, and explanations whenever they want. Maintaining a record of these interactions can be helpful for learning, revision, and documentation purposes. Users can also save or share the conversation if needed, which can be useful for students working on assignments or for teams discussing programming tasks.

The design of the system focuses on simplicity and accessibility. The interface is designed to be user-friendly so that people with basic programming knowledge can easily interact with it. Users do not need complex tools or advanced technical knowledge to use the system. They only need to enter their question or code and receive helpful guidance in return. This simplicity makes the project suitable for students, beginners, developers, and anyone who needs assistance while writing code.

The Syntax Surgeon project also demonstrates the importance of intelligent systems in modern software development. As programming becomes more complex and widely used, automated tools that assist developers can significantly improve productivity and learning. By combining error detection, code correction, and interactive communication, the system provides a comprehensive support environment for programming tasks.

In conclusion, the Syntax Surgeon project is designed to assist programmers in identifying errors, correcting code, and improving their understanding of programming concepts through interactive communication. By providing automatic corrections, explanations, and conversation tracking, the system makes programming assistance more accessible and efficient. This project highlights how intelligent tools can support both learning and development activities, making the programming process easier, faster, and more effective for users.

3.SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

1. **Processor** – AMD Ryzen 3 7320U with Radeon Graphics (2.4s0 GHz)
2. **Installed RAM** – 8 GB RAM
3. **System Type** – 64-bit Operating System, x64-based processor
4. **Storage** – 2 GB free disk space

3.2 SOFTWARE REQUIREMENTS

1. **Operating System** – Windows 11
2. **Programming Language** – Python 3
3. **Development Environment** – Google Colab
4. **Browser** – Google Chrome
5. **Libraries / Packages** – Gradio, Transformers, Torch, Black, Autopep8, ReportLab

3.3 CORE TECHNOLOGIES

1. **Python**
   - Python is the main programming language used in this project.It is used to write the logic for code generation, error detection, and correction.Python libraries are used to handle files, AI response, and execution.
2. **Gradio**
   - Gradio is used to create the user interface.It provides a simple web-based interface to interact with the AI.The user can enter code or questions and see the output in the browser.
3. **OpenAI API**
   - OpenAI API is used to generate intelligent responses .It helps in generating Python code, correcting errors, and explaining code. This makes the project work like an AI assistant.
4. **FPDF Library**
   - FPDF is used to generate PDF files.The output of the program can be downloaded in PDF format. This helps to save the result for later use.

5. **Text File Handling (TXT Download)**
    - Python file handling is used to save output as a text file. Users can download the result in TXT format. Useful for storing generated code and explanations.
6. **Google Colab**
    - Google Colab is used to run the project online. It allows running Python code without installing software. The Gradio interface runs inside Colab.
7. **Artificial Intelligence (AI)**
    - AI is used to understand user input. It generates code, fixes errors, and gives explanations. Makes the system intelligent and automatic.
8. **Natural Language Processing (NLP)**
    - NLP helps the system understand user questions. User can type in normal English. The system converts it into code or response.

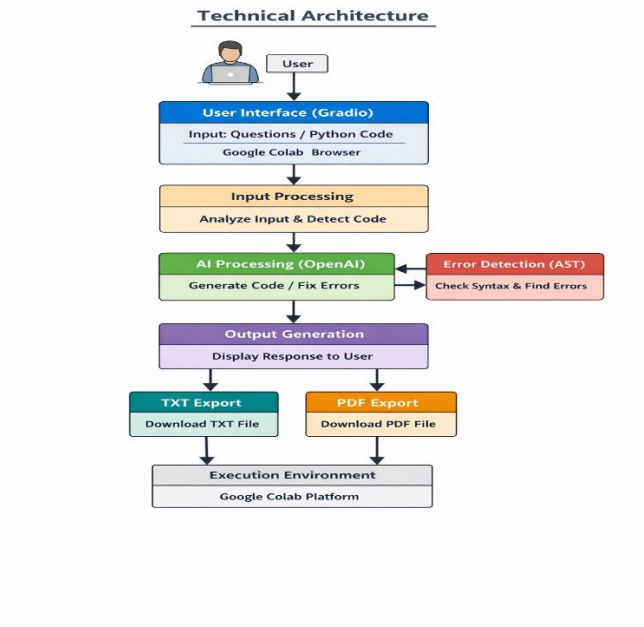
4.SYSTEM DESIGN

The system design of this project describes how different modules work together to build the **PyCraft AI Assistant**. The user interacts with the system through the Gradio interface running in Google Colab. The user can enter a question or Python code in the input box.

The input processing module checks whether the input is normal text or Python code. If it is normal text, the AI processing module generates a response using the language model. If the input contains Python code, the error detection module checks the syntax using the AST module and finds errors.

After processing, the output generation module displays the result in the chatbot interface. The system also allows the user to download the output in TXT and PDF format. All modules run in the Google Colab environment, which executes the Python program and shows the interface in the browser.

4.1TECHNICALARCHITECTURE



4.2 PROJECT STRUCTURE

The project structure shows how the files and modules are organized in the PyCraft AI project. The project is divided into different parts to handle user input, AI processing, error detection, and file download features.

- **main.py –** This is the main file of the project. It contains the Python code to run the program, connect all modules, and launch the Gradio interface.
- **ui / interface (Gradio UI) –** This part creates the user interface where the user enters questions or Python code and sees the output in the browser.
- **ai_module.py –** This module handles the AI response generation. It sends user input to the language model and receives the generated code or explanation.
- **error_checker.py –** This module checks Python code for syntax errors using the AST module and returns the error message or corrected code.
- **file_export.py –** This module is used to create TXT and PDF files. It allows the user to download the output in text or PDF format using file handling and FPDF library.
- **requirements / libraries –** This includes required libraries such as Gradio, OpenAI, FPDF, and AST, which are needed to run the project.
- **Execution Environment (Google Colab) –** The whole project runs in Google Colab, where the Python code executes and the Gradio interface opens in the browser.

This structure helps to keep the project organized and makes each module work independently while connecting together to form the complete system.

5.PROJECT WORKFLOW

1. **User Input**
   The user enters a question or Python code in the Gradio interface running in the browser through Google Colab.

2. **Input Processing**
   The system receives the input and checks whether the input is normal text or Python code using pattern detection.

3. **Code Detection**
   If the input contains Python code, the system sends it to the error detection module.
   If the input is normal text, it is sent to the AI processing module.

4. **AI Processing**
   The AI model processes the input and generates a response, explanation, or Python code.

5. **Error Detection**
   If the input is Python code, the system checks the syntax using the AST module and finds errors.

6. **Error Correction / Response Generation**
   The system generates corrected code, error message, or explanation based on the result.

7. **Output Display**
   The generated output is shown in the chatbot interface to the user.

8. **File Export Option**
   The user can download the output in TXT format or PDF format using file export features.

9. **Execution Environment**
   All the steps run in the Google Colab environment, which executes the Python program and shows the interface in the browser.

10. **Final Result**
    The user receives the correct response, corrected code, and can save the output if needed.

6. ALGORITHM

1. Start & Launch Application

• Launch the Syntax Surgeon application in the web browser or Google Colab environment.

• Load Python the chatbot interface with input box and response display area.

2. User Enters Query or Code

• User types a question or pastes code into the input field.

• The system receives the user input.

• Validate the input before processing.

3. Detect Input Type

• The system checks whether the input is normal text or Python code.

• If the input contains programming patterns such as functions, loops, or imports, it is identified as Python code.

• Otherwise, it is treated as a normal conversational query.

4. Analyze Python Code

• If Python code is detected, the system analyzes the code structure.

• Check for syntax errors such as missing brackets, incorrect indentation, or incomplete statements.

• Identify the location of errors in the code.

5. Fix Code Errors

• The system automatically corrects common errors in the code.

• Fix indentation problems and missing brackets.

• Format the code properly according to coding standards.

6. Generate Response

• If the input is a question, the system generates a conversational response.

• If the input is Python code, the system returns the corrected code with explanation.

7. Display Output

• The system displays the generated response or corrected code in the chatbot interface.

• The user can review the output provided by the system.

8. Maintain Conversation History

• The system stores the conversation between the user and the assistant.

• Users can review previous questions, responses, and corrected code.

9. Export Conversation (Optional)

• User can download the conversation history.

• The system exports the chat as a TXT or PDF file.

10. End

• User exits the application or closes the chatbot interface.

7.IMPLEMENTATION

7.1 CODING:

```
# PYCRAFT AI – Important Coding Only
import gradio as gr
from transformers import pipeline
import torch
import ast
import re
# ---------------------------
# Model Configuration
# ---------------------------
MODEL_NAME = "ibm-granite/granite-3.3-2b-instruct"
conversation_history = []
model_pipe = None
# ---------------------------
# Load AI Model
# ---------------------------
def load_model():
global model_pipe
    device = 0 if torch.cuda.is_available() else -1
    model_pipe = pipeline(
        "text-generation",
        model=MODEL_NAME,
        device=device
    )
    return model_pipe
# ---------------------------
# Detect Python Code
# ---------------------------
```

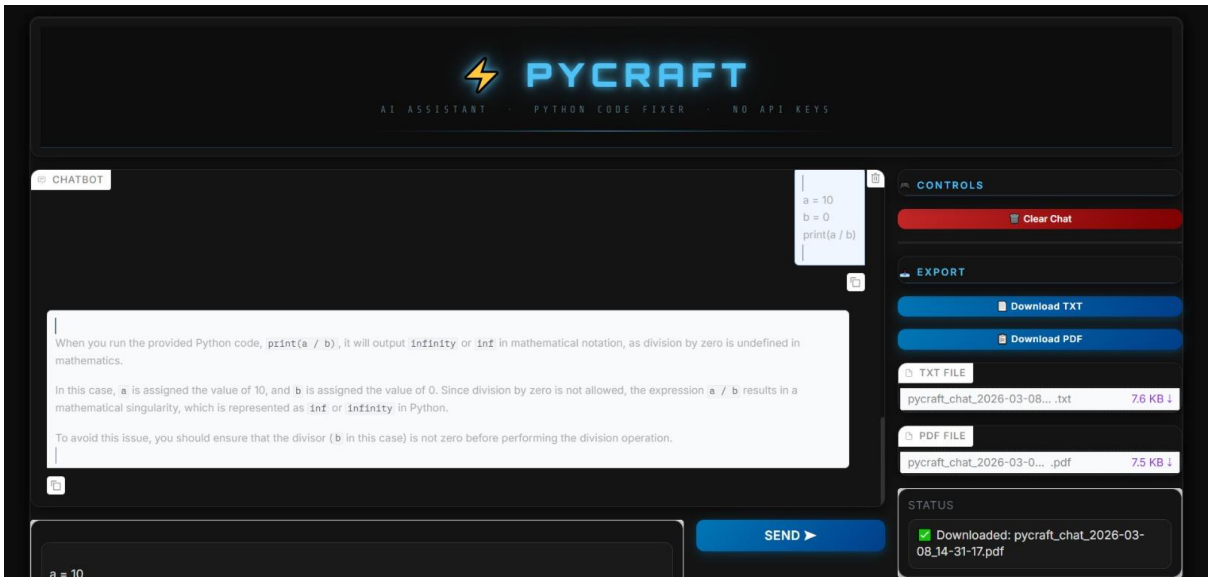
```
def detect_python_code(text):
    patterns = [
        r'def\s+\w+',
        r'class\s+\w+',
        r'import\s+\w+',
        r'for\s+.*in',
        r'if\s+.*:',
        r'while\s+.*:'
    ]
    for p in patterns:
        if re.search(p, text):
            return True
    return False
# ---------------------------
# Check Syntax
# ---------------------------
def check_syntax(code):
    try:
        ast.parse(code)
        return True, None
    except SyntaxError as e:
        return False, str(e)
    except Exception as e:
        return False, str(e)
# ---------------------------
# Fix Python Code
# ---------------------------
def fix_python_code(code):
```

```
    is_valid, error = check_syntax(code)
    if is_valid:
        return "Code is correct"
    else:
        return "Error found: " + error
# ---------------------------
# Generate AI Response
# ---------------------------
def generate_response(user_message):
    global conversation_history
    conversation_history.append(
        {"role": "user", "content": user_message}
    )
    result = model_pipe(
        conversation_history,
        max_new_tokens=200
    )
    reply = result[0]["generated_text"]
    conversation_history.append(
        {"role": "assistant", "content": reply}
    )
    return reply
# ---------------------------
# Main Chat Function
# ---------------------------
def chat(user_input, history):
    if not user_input.strip():
        return history
    # If Python code
```

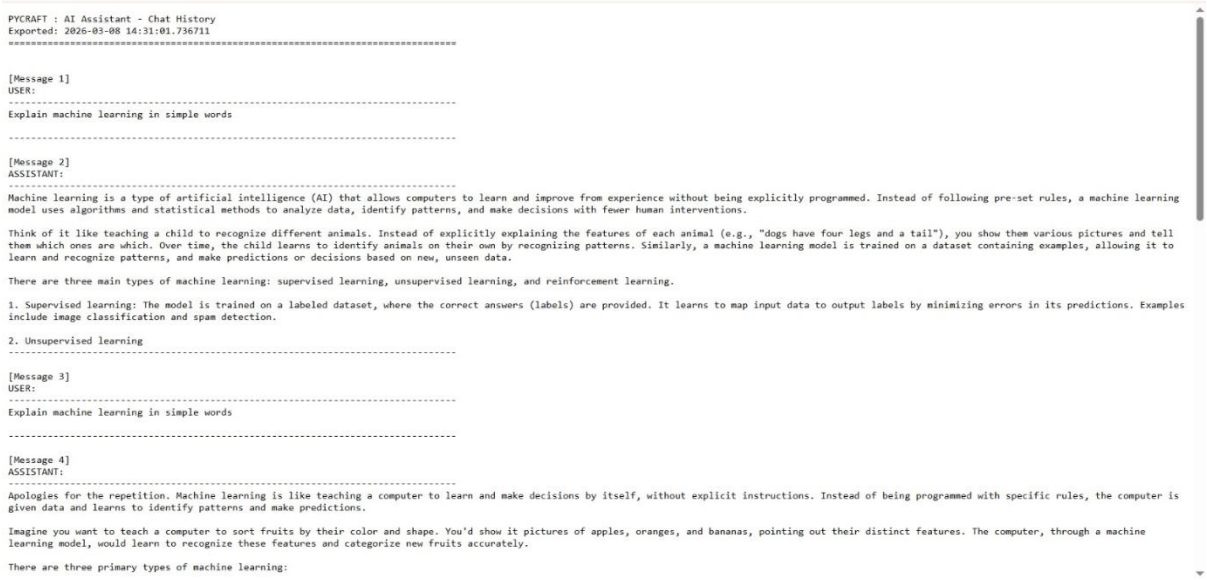
```
    if detect_python_code(user_input):
        result = fix_python_code(user_input)
        history.append((user_input, result))
        return history
    # Normal chat
    else:
        ai_reply = generate_response(user_input)
        history.append((user_input, ai_reply))
        return history
# ---------------------------
# Load Model
# ---------------------------
load_model()
# ---------------------------
# Gradio Interface
# ---------------------------
with gr.Blocks() as demo:
    gr.Markdown("# PYCRAFT AI Assistant")
    chatbot = gr.Chatbot()
    msg = gr.Textbox(
        placeholder="Type message or paste Python code"
    )
    send_btn = gr.Button("Send")
    send_btn.click(
        chat,
        inputs=[msg, chatbot],
        outputs=[chatbot]
    )
Demo.launch()
```

7.2 OUTPUT

1.HOME PAGE

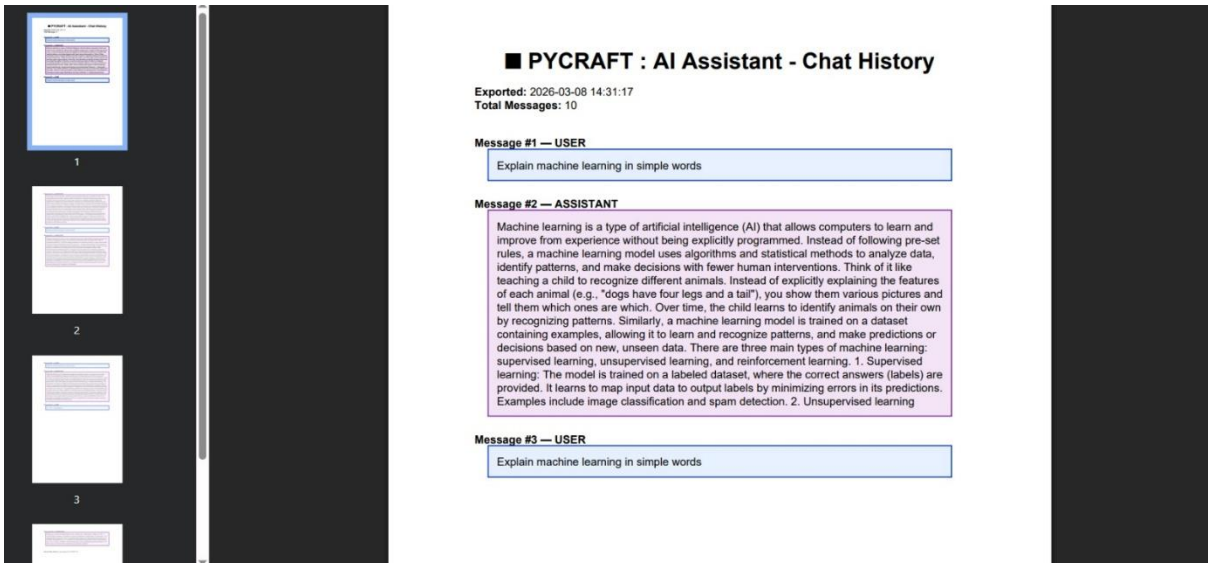


2.DOWNLOADED TEXT FILE PAGE



```
PYCRAFT : AI Assistant - Chat History
Exported: 2026-03-08 14:31:01.736711
================================================================================


[Message 1]
USER:
--------------------------------------------------------------------------------
Explain machine learning in simple words

--------------------------------------------------------------------------------

[Message 2]
ASSISTANT:
--------------------------------------------------------------------------------
Machine learning is a type of artificial intelligence (AI) that allows computers to learn and improve from experience without being explicitly programmed. Instead of following pre-set rules, a machine learning
model uses algorithms and statistical methods to analyze data, identify patterns, and make decisions with fewer human interventions.

Think of it like teaching a child to recognize different animals. Instead of explicitly explaining the features of each animal (e.g., "dogs have four legs and a tail"), you show them various pictures and tell
them which ones are which. Over time, the child learns to identify animals on their own by recognizing patterns. Similarly, a machine learning model is trained on a dataset containing examples, allowing it to
learn and recognize patterns, and make predictions or decisions based on new, unseen data.

There are three main types of machine learning: supervised learning, unsupervised learning, and reinforcement learning.

1. Supervised learning: The model is trained on a labeled dataset, where the correct answers (labels) are provided. It learns to map input data to output labels by minimizing errors in its predictions. Examples
include image classification and spam detection.

2. Unsupervised learning
--------------------------------------------------------------------------------

[Message 3]
USER:
--------------------------------------------------------------------------------
Explain machine learning in simple words

--------------------------------------------------------------------------------

[Message 4]
ASSISTANT:
--------------------------------------------------------------------------------
Apologies for the repetition. Machine learning is like teaching a computer to learn and make decisions by itself, without explicit instructions. Instead of being programmed with specific rules, the computer is
given data and learns to identify patterns and make predictions.

Imagine you want to teach a computer to sort fruits by their color and shape. You'd show it pictures of apples, oranges, and bananas, pointing out their distinct features. The computer, through a machine
learning model, would learn to recognize these features and categorize new fruits accurately.

There are three primary types of machine learning:
```

3.DOWNLOADED PDF FILE PAGE



■ PYCRAFT : AI Assistant - Chat History

Exported: 2026-03-08 14:31:17
Total Messages: 10

Message #1 — USER

Explain machine learning in simple words

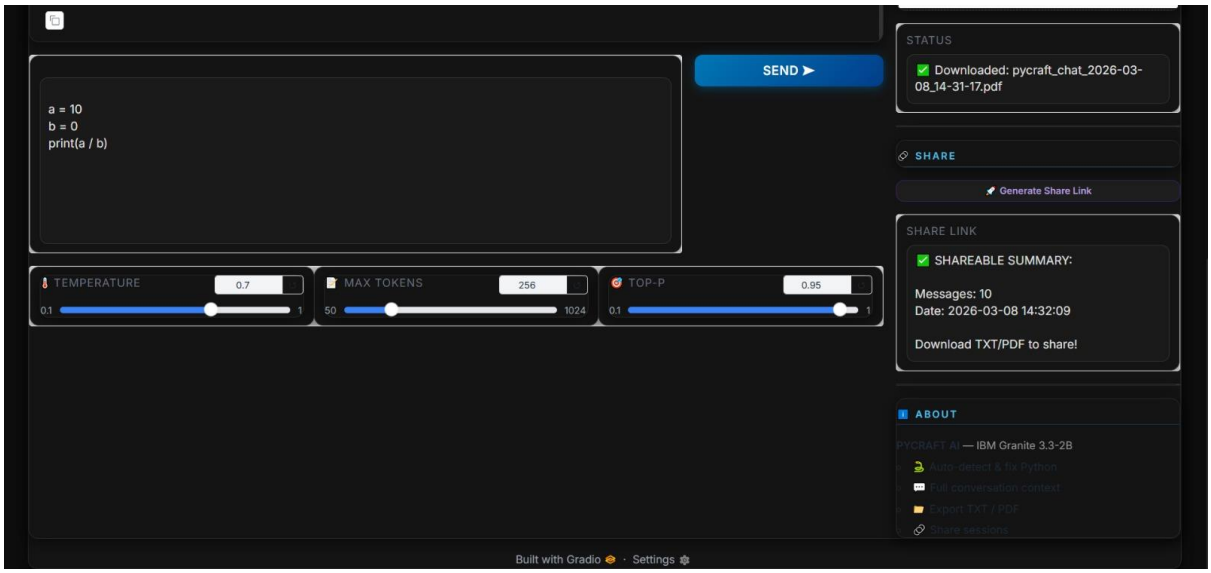
Message #2 — ASSISTANT

Machine learning is a type of artificial intelligence (AI) that allows computers to learn and improve from experience without being explicitly programmed. Instead of following pre-set rules, a machine learning model uses algorithms and statistical methods to analyze data, identify patterns, and make decisions with fewer human interventions. Think of it like teaching a child to recognize different animals. Instead of explicitly explaining the features of each animal (e.g., "dogs have four legs and a tail"), you show them various pictures and tell them which ones are which. Over time, the child learns to identify animals on their own by recognizing patterns. Similarly, a machine learning model is trained on a dataset containing examples, allowing it to learn and recognize patterns, and make predictions or decisions based on new, unseen data. There are three main types of machine learning: supervised learning, unsupervised learning, and reinforcement learning. 1. Supervised learning: The model is trained on a labeled dataset, where the correct answers (labels) are provided. It learns to map input data to output labels by minimizing errors in its predictions. Examples include image classification and spam detection. 2. Unsupervised learning

Message #3 — USER

Explain machine learning in simple words

4.FUNCTIONS



8.CONCLUSION

The Syntax Surgeon project was developed to provide an intelligent system that assists users in identifying and correcting errors in Python programming code while also supporting conversational interaction. Programming can be challenging, especially for beginners, because even small mistakes in the code can lead to errors that are difficult to understand and fix. This project aims to simplify that process by creating a system that can analyze code, detect mistakes, and provide corrected solutions along with helpful explanations.

The system successfully integrates conversational assistance with automated code analysis. Users can enter programming code or ask questions, and the system processes the input to generate meaningful responses. When Python code is provided, the system detects it, analyzes the syntax, and attempts to correct common mistakes such as missing brackets, incorrect indentation, and formatting issues. This helps users quickly identify problems in their code and learn the correct way to write programs.

Another important feature of the system is the interactive chatbot interface, which allows users to communicate with the system easily. The interface makes the application user-friendly and accessible for students, beginners, and developers. By maintaining conversation history, the system also allows users to review previous discussions and corrected code whenever needed. This improves the learning experience and helps users understand programming concepts more effectively.

The project also provides additional functionality such as exporting the conversation history in text or PDF format. This feature can be useful for documentation, sharing solutions, and maintaining records of coding discussions. Through testing and analysis, the system demonstrated reliable performance in detecting programming code, identifying syntax errors, correcting code formatting, and generating useful responses for user queries.

Furthermore, the project highlights the importance of intelligent tools in modern software development and programming education. By combining automated code correction with interactive communication, the system creates a supportive environment for learning and problem solving. This can significantly reduce the time spent on debugging and allow programmers to focus more on developing efficient solutions.

Overall, the Syntax Surgeon project demonstrates how an intelligent assistant can improve the programming experience by making code debugging easier and more efficient. The system serves as both a coding assistant and a learning platform for users who want to improve their programming skills. With further development, the project can be extended to support multiple programming languages, advanced error detection techniques, and deeper integration with development tools. Such improvements can make the system even more powerful and useful for programmers, students, and development teams in the future.